

EC2 Fleet Standardization Automation

Designing a Safe, Repeatable Infrastructure Change Workflow on AWS

Role

Cloud / DevOps Engineer

Project Type

Production-Inspired Infrastructure Automation

Technology Stack

AWS • Python • Boto3 • Terraform • EC2 • IAM • YAML • Git • Linux

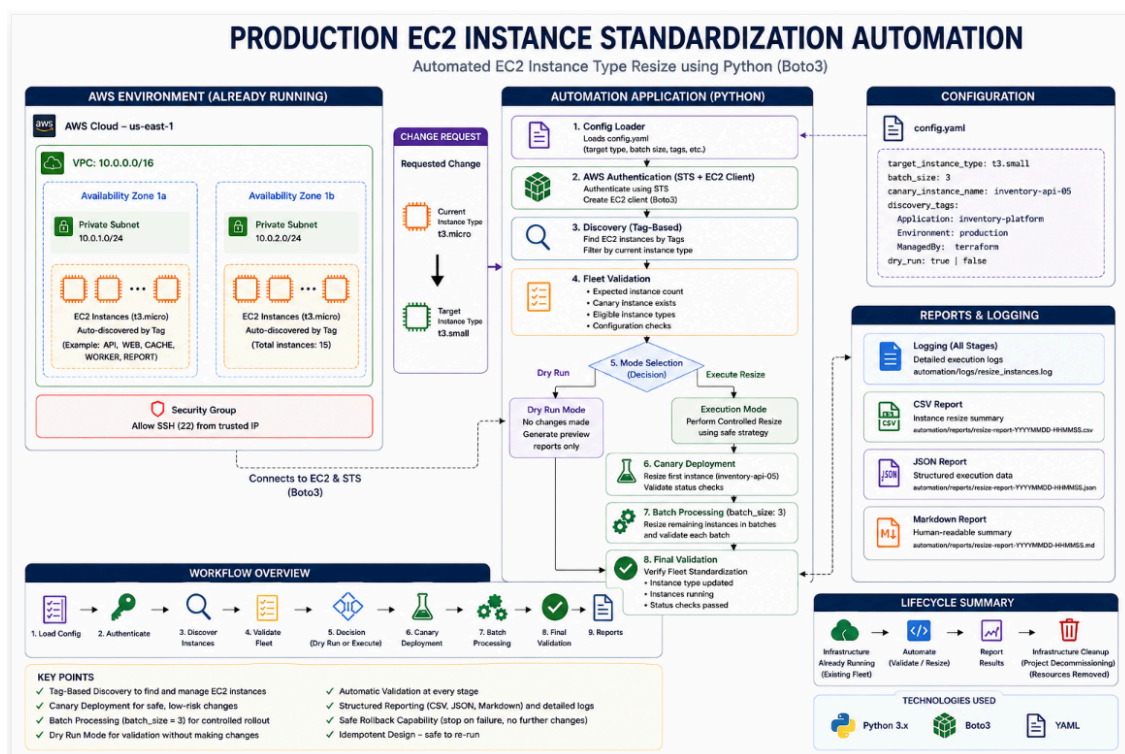


Figure 1. High-level architecture illustrating the complete infrastructure standardization workflow.

Architecture Goals

The architecture was designed around five objectives:

- Separate infrastructure provisioning from operational automation.
- Reduce operational risk through staged execution.
- Detect failures before infrastructure changes begin.
- Produce consistent execution results across repeated runs.
- Generate detailed reports for auditing and troubleshooting.

Executive Summary

Cloud infrastructure changes are rarely difficult because engineers cannot call an API. More often, failures occur because changes are executed inconsistently, assumptions go unvalidated, or insufficient safeguards are in place before modifications begin.

I built this project to automate the standardization of an EC2 fleet by upgrading fifteen Amazon EC2 instances from **t3.micro** to **t3.small** using Python and AWS Boto3. Rather than focusing solely on automating the resize operation, I designed the workflow around the principles I believe are essential for safe infrastructure management: validating assumptions, limiting operational risk, maintaining execution visibility, and producing repeatable outcomes.

The automation follows a staged execution model that validates the environment before making changes, performs a canary deployment to reduce the blast radius of potential failures, executes the remaining workload in controlled batches, verifies the final infrastructure state, and generates detailed execution reports for operational review.

The result is a repeatable workflow that reduces manual effort while providing predictable infrastructure changes and complete execution traceability.

Project Highlights

- Automated the standardization of 15 EC2 instances from t3.micro to t3.small
 - Built a modular automation framework using Python and AWS Boto3
 - Implemented canary deployment to reduce deployment risk
 - Added automated validation before infrastructure changes
 - Generated structured CSV, JSON, and Markdown execution reports
 - Used Terraform to provision and destroy infrastructure consistently
 - Designed with modular components to simplify maintenance and future enhancements
-

The Problem

As cloud environments grow, repetitive infrastructure tasks quickly become operational challenges. Even a simple EC2 resize becomes increasingly difficult when the same change must be performed consistently across multiple instances while maintaining visibility into every modification.

A manual workflow introduces several risks:

- Human error during repetitive operations
- Inconsistent execution between environments
- Longer maintenance windows
- Limited operational visibility
- Poor auditability
- Configuration drift over time

Instead of creating another EC2 resize script, I wanted to design an automation workflow that approached infrastructure changes as controlled engineering operations.

My Design Philosophy

Before writing any code, I defined a set of engineering principles that shaped the design of this automation. These principles influenced every major decision, from how the workflow validates its environment to how infrastructure changes are executed and verified. Rather than optimizing for the shortest implementation, I focused on building a solution that is predictable, maintainable, and operationally safe.

Safety Before Speed

I deliberately prioritized safe execution over reducing deployment time. Infrastructure changes are easier to recover from when they are performed in a controlled and predictable manner. Features such as staged validation, canary deployment, and controlled batch execution were implemented to reduce operational risk, even though they increase the overall execution time.

Fail Fast

The workflow validates AWS credentials, configuration files, and infrastructure state before attempting any modifications. Detecting problems early prevents partially completed infrastructure changes and provides immediate feedback when the execution environment is not ready.

Minimize Blast Radius

Instead of upgrading the entire fleet at once, I introduced a canary deployment strategy that validates the workflow on a single EC2 instance before continuing with the remaining fleet. This approach limits the impact of unexpected failures and provides confidence before wider deployment.

Single Responsibility

Each module was designed to perform a single well-defined function, such as discovery, validation, execution, logging, or reporting. Separating responsibilities improves readability, simplifies testing, and makes the project easier to maintain as new capabilities are added.

Observability

Every execution generates structured logs and reports that capture validation results, infrastructure changes, timestamps, and execution status. Maintaining these execution artifacts provides a reliable audit trail and makes post-deployment troubleshooting significantly easier than relying solely on terminal output.

Solution Overview

Instead of implementing one large automation script, I divided the workflow into independent stages.

Stage	Responsibility
Authentication	Verify AWS identity and permissions
Configuration Validation	Validate execution parameters
Fleet Discovery	Dynamically identify target EC2 instances
Safety Validation	Ensure every instance is eligible for modification
Canary Deployment	Validate the workflow on a single instance
Batch Execution	Upgrade the remaining fleet in controlled batches
Verification	Confirm the expected infrastructure state
Reporting	Produce execution artifacts for operational review

Separating these responsibilities made the automation easier to troubleshoot, maintain, and extend.

Why I Chose This Architecture

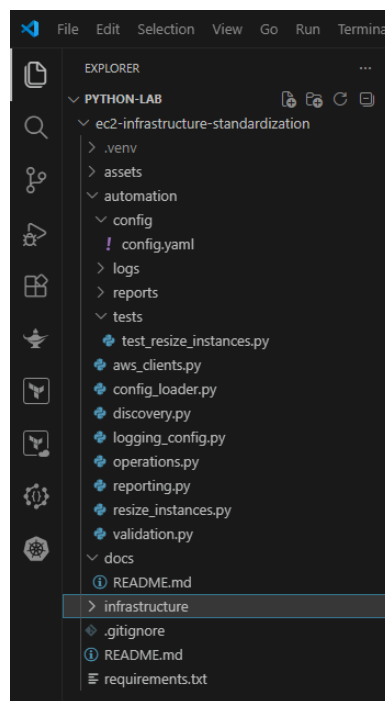


Figure 2. Repository structure showing separation between infrastructure provisioning, automation modules, configuration, and reporting.

One of the earliest architectural decisions was separating infrastructure provisioning from operational automation.

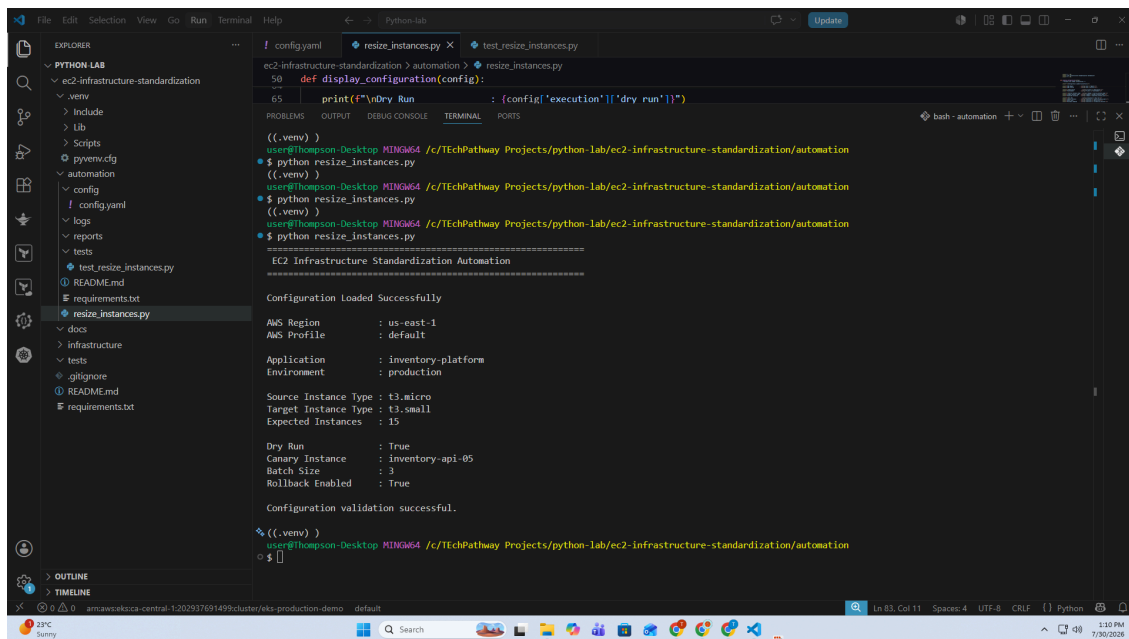
Terraform is responsible for provisioning and destroying infrastructure, while Python orchestrates the operational workflow using AWS Boto3.

Within the Python application, discovery, validation, execution, logging, and reporting are implemented as independent modules rather than a single script. Although this introduced additional project structure, it improved maintainability, simplified testing, and made future enhancements significantly easier.

Technology Decisions

Although the workflow could have been implemented using Bash and the AWS CLI, Python provided a better foundation for building a modular application with reusable components, structured exception handling, configuration management, and report generation. As the project evolved beyond a simple resize script, Python made the solution significantly easier to maintain and extend.

Execution Workflow



```
def display_configuration(config):
    print(f"\nDry Run : {config['execution']['dry_run']}")

((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

=====
EC2 Infrastructure Standardization Automation
=====

Configuration Loaded Successfully

AWS Region      : us-east-1
AWS Profile     : default
Application     : inventory-platform
Environment     : production

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances  : 15

Dry Run          : True
Canary Instance  : inventory-api-05
Batch Size       : 3
Rollback Enabled : True

Configuration validation successful.

((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$
```

Figure 3. Configuration successfully validated before execution.

The automation begins by validating the execution environment rather than immediately interacting with AWS resources.

Configuration files, AWS credentials, execution parameters, and runtime options are verified before any API calls that modify infrastructure are performed.

Failing early reduces troubleshooting complexity and prevents partially completed workflows caused by invalid configuration.

```

((.venv) )
user@Thompson-Desktop MINGW64 /c/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

=====
EC2 Infrastructure Standardization Automation
=====

Configuration Loaded Successfully

AWS Region      : us-east-1
AWS Profile     : default

Application     : inventory-platform
Environment     : production

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances  : 15

Dry Run        : True
Canary Instance : inventory-api-05
Batch Size     : 3
Rollback Enabled : True

Configuration validation successful.

Connecting securely to AWS...
2026-07-30 20:05:02,301 | INFO | Successfully authenticated with AWS.
AWS connection successful.
AWS Account ID      : 202937691499
EC2 Region         : us-east-1

Discovering EC2 instances using configured tags...
2026-07-30 20:05:03,149 | INFO | Discovered 15 matching EC2 instances.

Discovery Summary
-----
Expected Instances : 15
Discovered Instances : 15
Discovery Status   : PASS
-----

  Name                Role    Type    State    Availability Zone
-----
inventory-api-01      api     t3.micro running us-east-1a
inventory-api-02      api     t3.micro running us-east-1b
inventory-api-03      api     t3.micro running us-east-1a
inventory-api-04      api     t3.micro running us-east-1b
inventory-api-05      api     t3.micro running us-east-1a
inventory-cache-01     cache   t3.micro running us-east-1a
inventory-cache-02     cache   t3.micro running us-east-1b
inventory-report-01    reporting t3.micro running us-east-1a
inventory-report-02    reporting t3.micro running us-east-1b

```

Figure 4. Dynamic discovery of EC2 instances matching the configured selection criteria.

Instead of relying on manually maintained inventory lists, the automation dynamically discovers target instances using predefined selection criteria. This reduces configuration drift while ensuring the workflow always operates against the current infrastructure state.

```

((.venv) )
user@Thompson-Desktop MINGW64 /c/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances  : 15

Dry Run        : True
Canary Instance : inventory-api-05
Batch Size     : 3
Rollback Enabled : True

Configuration validation successful.

Connecting securely to AWS...
2026-07-30 16:42:03,989 | INFO | Successfully authenticated with AWS.
AWS connection successful.
AWS Account ID      : 202937691499
EC2 Region         : us-east-1

Discovering EC2 instances using configured tags...

Discovery Summary
-----
Expected Instances : 15
Discovered Instances : 0
Discovery Status   : FAILED
-----

Fleet Validation
-----
Validation Status : FAILED

The automation cannot continue:
- Expected 15 instances, but discovered 0.
- Configured canary instance 'inventory-api-05' was not discovered.

No EC2 changes were performed.

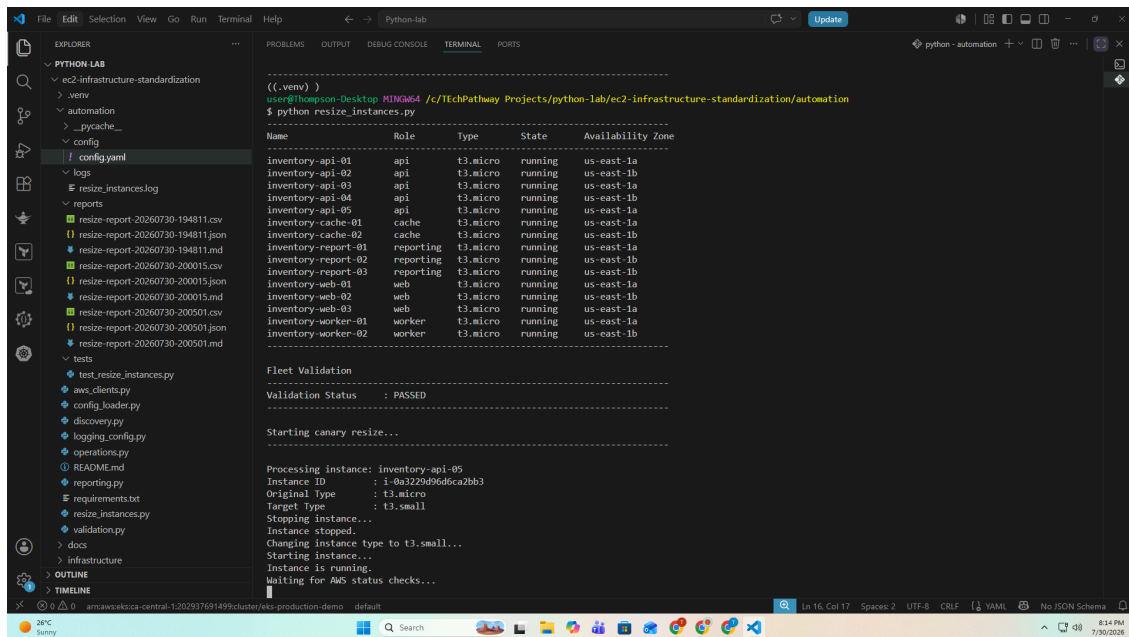
% ((.venv) )
user@Thompson-Desktop MINGW64 /c/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation

```

Figure 5. Validation confirming the fleet is ready before infrastructure modifications begin.

Before modifying any instance, the workflow validates current instance type, power state, and execution prerequisites.

If any validation fails, execution terminates before infrastructure changes begin.



```
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

Name      Role      Type      State      Availability Zone
-----
inventory-api-01  api      t3.micro   running    us-east-1a
inventory-api-02  api      t3.micro   running    us-east-1b
inventory-api-03  api      t3.micro   running    us-east-1a
inventory-api-04  api      t3.micro   running    us-east-1b
inventory-api-05  api      t3.micro   running    us-east-1a
inventory-cache-01 cache     t3.micro   running    us-east-1a
inventory-cache-02 cache     t3.micro   running    us-east-1b
inventory-report-01 reporting t3.micro   running    us-east-1a
inventory-report-02 reporting t3.micro   running    us-east-1b
inventory-report-03 reporting t3.micro   running    us-east-1b
inventory-web-01  web      t3.micro   running    us-east-1a
inventory-web-02  web      t3.micro   running    us-east-1b
inventory-web-03  web      t3.micro   running    us-east-1a
inventory-worker-01 worker    t3.micro   running    us-east-1a
inventory-worker-02 worker    t3.micro   running    us-east-1b

Fleet Validation
-----
Validation Status : PASSED

Starting canary resize...

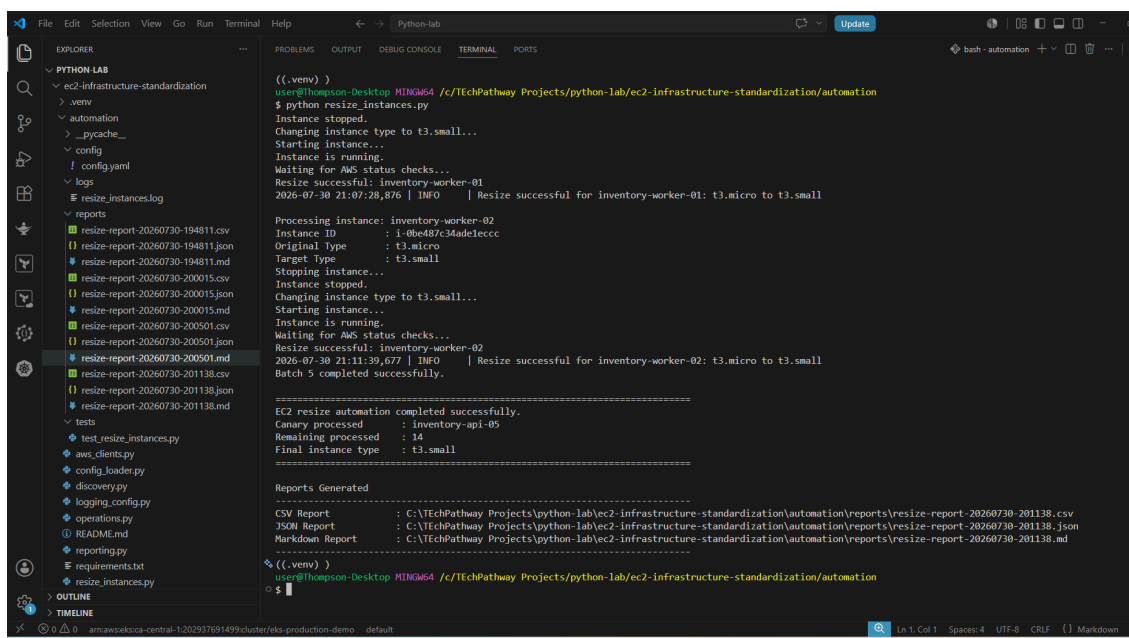
Processing Instance: inventory-api-05
Instance ID      : i-0a3229b6d6ca2bb3
Original Type    : t3.micro
Target Type      : t3.small
Stopping instance...
Instance stopped.
Changing instance type to t3.small...
Starting instance...
Instance is running.
Waiting for AWS status checks...
```

Figure 6. Canary deployment validating the workflow on a single EC2 instance.

Rather than upgrading all instances simultaneously, I implemented a canary deployment strategy.

The workflow upgrades one instance first, verifies successful completion, and only then proceeds with the remaining fleet.

This significantly reduces the blast radius of unexpected failures.



```
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

Instance stopped.
Changing instance type to t3.small...
Starting instance...
Instance is running.
Waiting for AWS status checks...
Resize successful: inventory-worker-01
2026-07-30 21:07:28,876 | INFO | Resize successful for inventory-worker-01: t3.micro to t3.small

Processing instance: inventory-worker-02
Instance ID      : i-0be487c34ade1ecc
Original Type    : t3.micro
Target Type      : t3.small
Stopping instance...
Instance stopped.
Changing instance type to t3.small...
Starting instance...
Instance is running.
Waiting for AWS status checks...
Resize successful: inventory-worker-02
2026-07-30 21:11:39,677 | INFO | Resize successful for inventory-worker-02: t3.micro to t3.small
Batch 5 completed successfully.

=====
EC2 resize automation completed successfully.
Canary processed      : inventory-api-05
Remaining processed   : 14
Final instance type    : t3.small

=====
Reports Generated
-----
CSV Report      : C:\TechPathway Projects\python-lab\ec2-infrastructure-standardization\automation\reports\resize-report-20260730-201138.csv
JSON Report     : C:\TechPathway Projects\python-lab\ec2-infrastructure-standardization\automation\reports\resize-report-20260730-201138.json
Markdown Report : C:\TechPathway Projects\python-lab\ec2-infrastructure-standardization\automation\reports\resize-report-20260730-201138.md

((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$
```

Figure 7. Successful controlled batch execution across the remaining EC2 instances.

Following successful canary validation, the remaining instances are upgraded in controlled batches.

Although sequential execution increases total runtime, I intentionally accepted this trade-off because it simplifies troubleshooting, improves execution visibility, and reduces operational risk.

Failure Handling Strategy

The workflow intentionally fails before infrastructure modifications begin whenever the environment does not satisfy predefined operational requirements.

Examples include:

- Authentication cannot be established.
- Configuration values are incomplete or inconsistent.
- Target instances cannot be discovered.
- Fleet validation fails.
- Canary deployment is unsuccessful.

Failing early prevents partially completed infrastructure changes and simplifies recovery by ensuring that no modifications occur until the environment is fully validated.

Reporting and Auditability

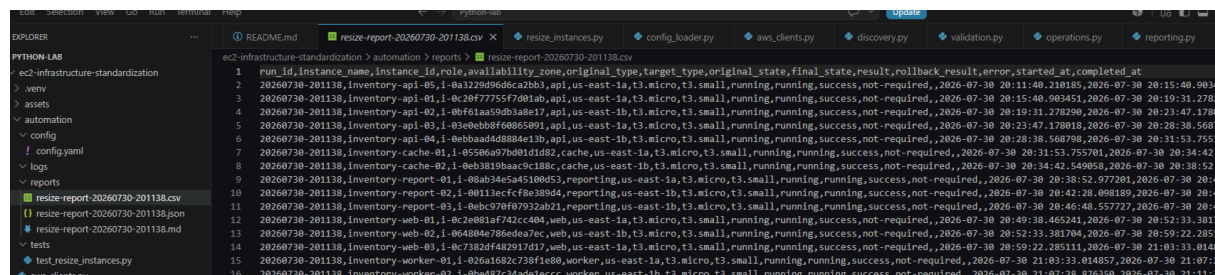


Figure 8. Structured execution reports generated after every automation run.

One design decision I considered particularly important was preserving execution artifacts after every run.

Console output is transient and difficult to reference during operational reviews. By automatically generating CSV, JSON, and Markdown reports, every execution produces a permanent record of the infrastructure state, validation results, timestamps, and execution outcome.

This greatly simplifies troubleshooting while also providing a historical audit trail that can be reviewed without rerunning the automation.

Project Metrics

Metric	Value
EC2 Instances Standardized	15
Deployment Strategy	Canary + Controlled Batches
Programming Language	Python
AWS SDK	Boto3
Infrastructure Provisioning	Terraform
Report Formats	CSV, JSON, Markdown
Deployment Risk	Reduced through staged execution

Engineering Trade-offs

Several design decisions intentionally favored operational safety over execution speed.

Decision	Trade-off
Canary deployment	Longer execution time, significantly lower deployment risk
Sequential batches	Slower execution, easier troubleshooting
Modular architecture	More project structure, better maintainability
Structured reporting	Additional storage, improved auditability
Fail-fast validation	Early termination, greater infrastructure consistency

These decisions intentionally prioritize operational safety, maintainability, and predictability over raw execution speed. While faster implementations are possible, I believe reducing operational risk is the more appropriate design choice for infrastructure automation.

Future Enhancements and Reflection

Developing this project reinforced that successful infrastructure automation extends beyond writing scripts. The most valuable engineering effort was spent designing validation logic, separating responsibilities, reducing operational risk, and ensuring every infrastructure change could be executed consistently.

If this automation were adopted within a larger engineering environment, I would enhance it with centralized logging, automated rollback capabilities, configurable parallel execution, CloudWatch dashboards, and event-driven notifications through Amazon SNS or Slack.

I would also integrate automated testing into a CI/CD pipeline to validate workflow changes before deployment and improve deployment confidence.

More importantly, this project reinforced that successful infrastructure automation is not simply about automating cloud API calls. It is about making thoughtful engineering decisions that improve reliability, consistency, and operational visibility.